

Tercero de Ingeniería Biomédica
Análisis de Datos Clínicos

Predicción de la readmisión
hospitalaria de pacientes con
diabetes

Curso 2025-2026

Víctor Sánchez Auñón

Índice:

1. Introducción
 - 1.1. Motivación.
 - 1.2. Objetivo del trabajo
2. Dataset y problema
3. Preprocesamiento
4. Análisis Exploratorio (EDA)
 - 4.1. Cuantificación de la variable objetivo
 - 4.2. Distribuciones de las categorías importantes
 - 4.3. Baseline (DummyClassifier)
5. Modelado inicial
 - 5.1. K-Nearest Neighbors
 - 5.2. Regresión logística
6. Gestión del desbalance, SMOTE
7. Modelado intermedio
 - 7.1. DecisionTreeClassifier
 - 7.2. RandomForestClassifier
 - 7.3. ADABOOST
8. Selección de características
 - 8.1. Algoritmos de selección de características
 - 8.2. Discusión de resultados
9. Optimización de hiperparámetros
 - 9.1. GridSearchCV
 - 9.2. Optuna
 - 9.3. Resultados y discusión
10. Modelado avanzado
11. Comparación estadística
12. Explicabilidad (XAI)
 - 12.1. SHAP
 - 12.2. LIME
13. Análisis de errores
14. Fairness
15. Conclusiones
16. Bibliografía

1. Introducción

1.1. Motivación.

La readmisión hospitalaria temprana es un grave problema para la gestión hospitalaria que perjudica al sistema por los costes monetarios, de recursos y la saturación de los centros. Además, estos reingresos perjudican directamente a los pacientes, ya que les interrumpe la recuperación, causa estrés y puede aumentar la exposición a infecciones nosocomiales.

Las causas más comunes de estos reingresos evitables son errores en la medicación, falta de apoyo domiciliario o no reconocer señales de advertencia. Teniendo también un mayor factor de riesgo pacientes con enfermedades como la insuficiencia cardíaca, patologías crónicas o factores sociales desfavorables [1].

1.2. Objetivo del trabajo

Por los anteriores motivos, para contribuir a la reducción de las readmisiones, en este proyecto se realizará un modelo de aprendizaje para la predicción de una readmisión temprana (<30 días) de pacientes con diabetes tras el alta.

Para ello, se usará un dataset gratuito de UC Irvine Machine Learning Repository con el que se hará primero un preprocesamiento de la información, un análisis y después se entrenará modelos de aprendizaje y se hará una comparación de sus rendimientos.

2. Dataset y problema

El dataset que se usará es *Diabetes 130-US Hospitals for Years 1999-2008* [2], que recoge 47 columnas de cada ingreso de un paciente con sus datos, medicación y diagnósticos entre otros, recogidos de 130 hospitales de Estados Unidos durante 10 años.

El dataset incluye un total de 101766 instancias, siendo la variable objetivo *readmitted*, la cual está dividida en 3 clases: NO, para pacientes que no han sido readmitidos, >30 para los que han vuelto en más de 30 días y <30 para los que han sido readmitidos en menos de 30 días.

Para reducir la complejidad del análisis, se ha transformado en un problema de clasificación binaria, donde la clase positiva (1) corresponde a pacientes readmitidos en menos de 30 días (<30), mientras que la clase negativa (0) agrupa tanto a los pacientes no readmitidos (NO) como a aquellos readmitidos después de más de 30 días (>30).

Además, se observa un claro desbalance en la distribución de clases, siendo la clase de readmisión temprana significativamente minoritaria en comparación con la clase negativa, lo cual se comprobará más tarde.

3. Preprocesamiento

Como se ha mencionado anteriormente, primero se realizó una conversión de la variable objetivo a una clasificación binaria y después se han codificado valores faltantes como “?” o *None* a NaN de la librería *numpy*, ya que de esta forma se podrá manejar dichos valores faltantes.

Al primer vistazo del dataset después de hacer uso del método `.info()`, se observa un número elevado de estos datos faltantes en ciertas columnas con porcentajes extremadamente altos de estos.

En concreto, se detectó que la variable *weight* contenía aproximadamente un 97% de valores ausentes, *max_glu_serum* en torno a un 95%, y *AICresult* sobre 83%. Además, variables como *medical_specialty* y *payer_code* presentaban aproximadamente un 50% y 60% de valores ausentes respectivamente.

Debido a estos porcentajes, se decidió eliminar estas variables ya que su aportación introduce ruido para los modelos de predicción, además, se eliminaron *encounter_id* y *patient_nbr* por ser simplemente valores identificativos de la fila del dataset y del paciente que no aportan información.

A continuación, se procede al tratamiento de las variables categóricas del conjunto de datos, con el objetivo de convertirlas a un formato numérico que pueda ser interpretado por los modelos de aprendizaje automático.

En el caso de variables categóricas sin orden como *race* o *gender*, se ha aplicado la técnica de one-hot encoding, generando variables binarias para cada una de las categorías existentes. Permitiendo al algoritmo entender la presencia o ausencia de un dato sin asumir un orden jerárquico erróneo [3].

Después se trataron las variables *diag_1*, *diag_2* y *diag_3*. Estas corresponden a los diagnósticos primarios, secundarios y terciarios respectivamente, codificados como los primeros tres dígitos del código ICD-9. El sistema ICD-9 (*International Classification of Diseases*) es una clasificación médica estandarizada utilizada para codificar diagnósticos y procedimientos clínicos de forma estructurada, permitiendo su uso en entornos hospitalarios y de investigación.

Al presentar una gran cardinalidad, se optó por recodificar la variable en 9 categorías sistémicas como por ejemplo circulatorio, respiratorio o digestivo para posteriormente aplicar one-hot encoding.

Al igual que la forma anterior, se realizó el mismo procedimiento para las variables *admission_type_id*, *discharge_disposition_id* y *admission_source_id*, ya que presentan una gran cantidad de valores distintos. En este caso, se agruparon en categorías más generales con significado clínico, relacionadas con el tipo de ingreso del paciente, el destino tras el alta y el origen de la admisión hospitalaria respectivamente.

Además, para la variable *discharge_disposition_id* se eliminaron las filas correspondientes a códigos relacionados con pacientes en tratamientos paliativos, debido a que estos pacientes van a fallecer y no aportan información al modelo en el contexto de las readmisiones.

Este tipo de separaciones reducen la complejidad y mejora la interpretabilidad del modelo gracias a la reducción de la cardinalidad de las variables originales y a la sustitución de códigos numéricos sin significado clínico directo por categorías más generales.

Finalmente, se mapearon las variables categóricas que sí contenían un orden, como por ejemplo la edad (*age*) o los medicamentos.

Para la primera variable que estaba dispuesta en rangos de edad de diez en diez, se transformaron a un valor medio, por ejemplo el rango [30-40) sería 35.

En cuanto a los medicamentos, que ocupan una gran cantidad de las columnas de estos datos, pueden tomar los valores *up* si la dosis fue incrementada, *down* si fue reducida, *steady* si no hubo cambios y *no* si el fármaco no fue prescrito. Para procesarlos, primero se comprobó con el método *value_counts()* normalizado el porcentaje de cada clase, encontrando algún fármaco como el *citoglipton* que contenía sólo valores *no*.

Dado este comportamiento, se eliminaron aquellos medicamentos con poca representación ya que no aportan capacidad discriminativa al modelo. Estos medicamentos son *repaglinide*, *nateglinide*, *chlorpropamide*, *acetohexamide*, *tolbutamide*, *acarbose*, *miglitol*, *troglitazone*, *tolazamide*, *examide*, *citoglipton*, *glyburide-metformin*, *glipizide-metformin*, *glimepiride-pioglitazone*, *metformin-rosiglitazone* y *metformin-pioglitazone*.

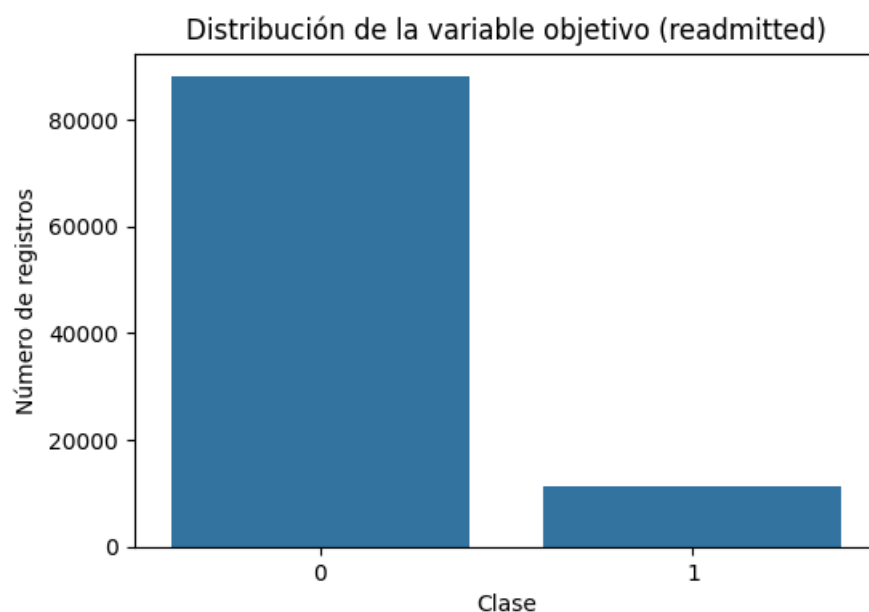
Para el resto de variables relacionadas con la medicación, se procedió a su codificación numérica asignando un valor distinto a cada una de las categorías posibles (*no*, *steady*, *up*, *down*), permitiendo su utilización directa por los algoritmos de aprendizaje automático. Asimismo, las variables generadas

mediante one-hot encoding fueron convertidas de valores booleanos a enteros (0 y 1) para una mejor comprensión de los modelos de aprendizaje.

4. Análisis Exploratorio (EDA)

4.1. Cuantificación de la variable objetivo

Con el dataset preprocesado, primero observaremos la ya mencionada clase target desbalanceada, al hacer un estudio de esta se observa un claro desbalance, teniendo en la clase 0 (No readmitidos en <30 días) 88026 registros (88.61% del total) y de los readmitidos 11314 registros (11.39%) como se puede apreciar en la Figura:



s

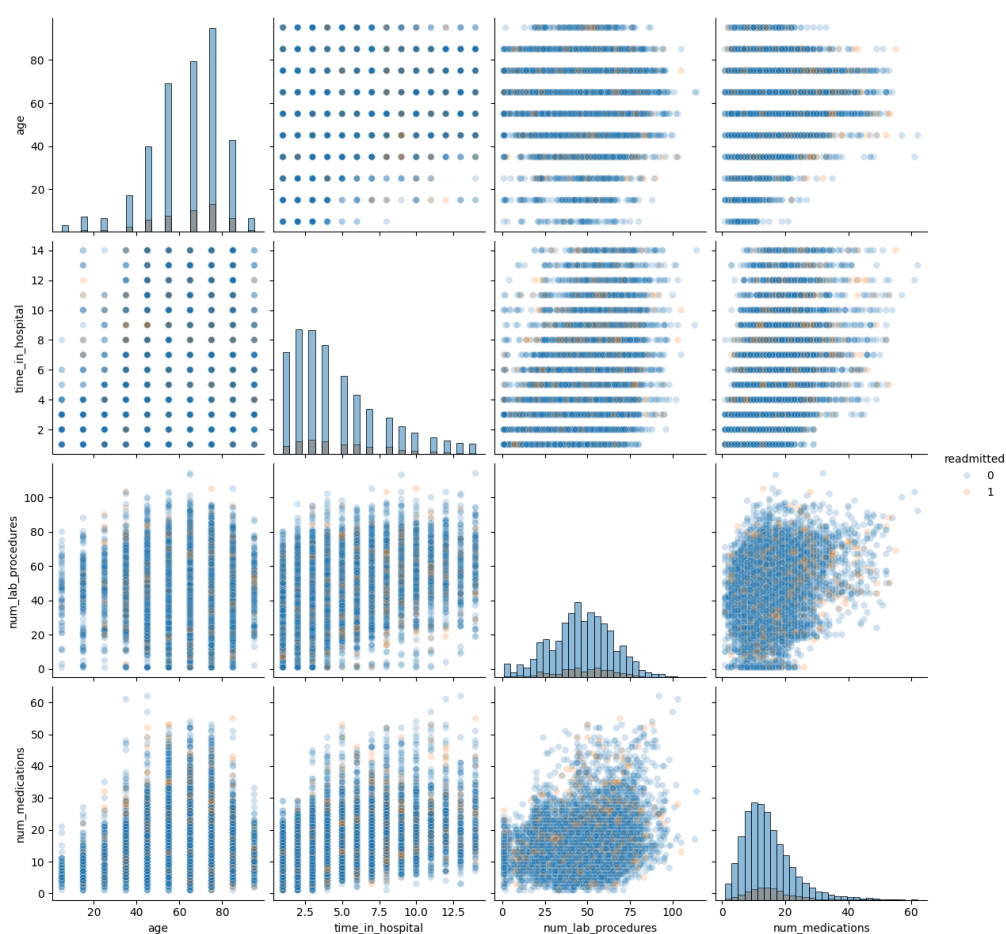
Dada la anterior distribución, se usarán principalmente métricas para evaluar los modelos como la precisión equilibrada (“balanced accuracy”) y el AUC-ROC, que permiten comprobar el desempeño de los modelos evitando predicciones sesgadas hacia la clase dominante.

Asimismo, durante el entrenamiento de los modelos implementados con la librería Scikit-learn, se empleará el parámetro “class_weight=‘balanced’” en aquellos algoritmos que lo permitan, con el objetivo de compensar el desbalance presente en los datos y penalizar de forma adecuada los errores en la clase minoritaria, que es la de mayor interés clínico.

4.2. Distribuciones de las categorías importantes

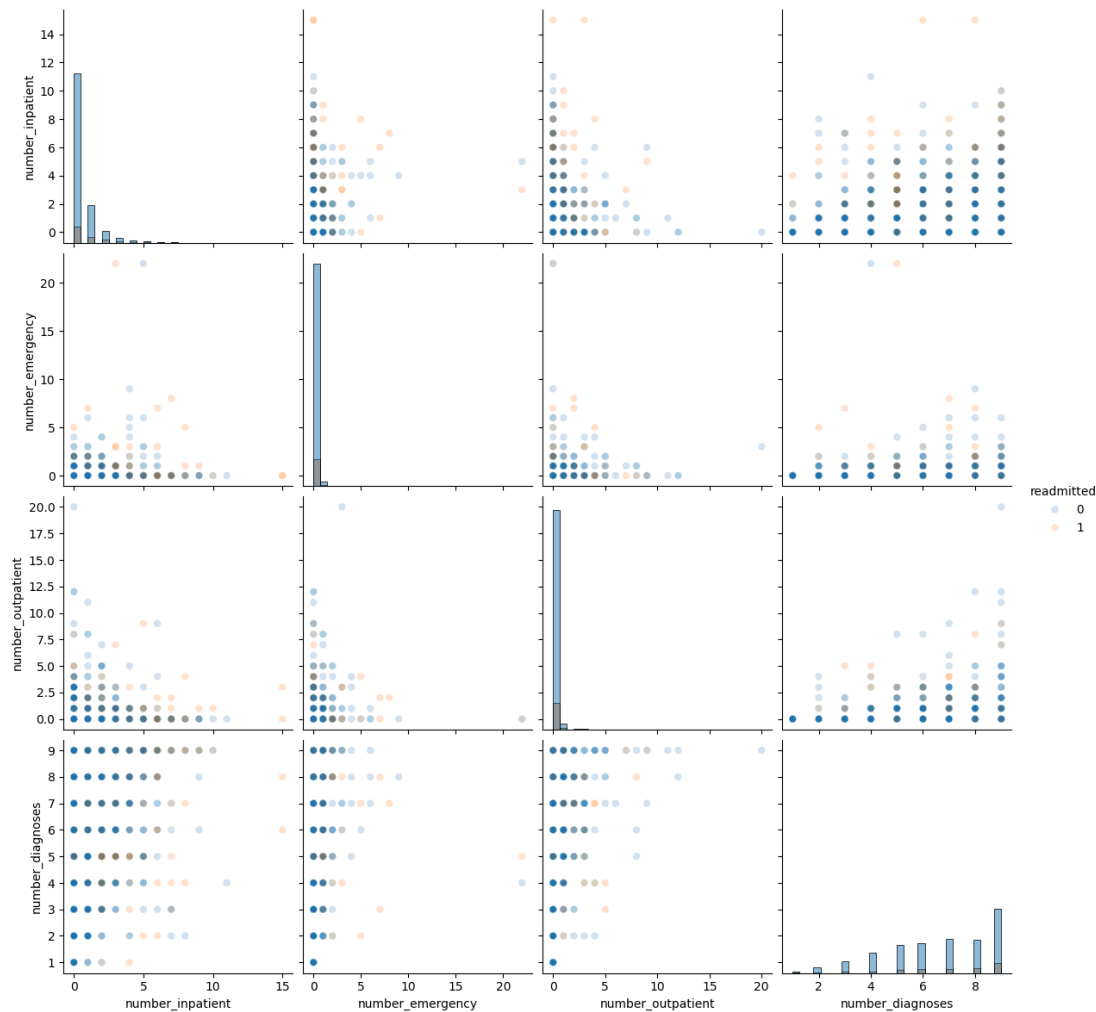
A continuación se realiza un análisis exploratorio de las variables más relevantes para la predicción de la readmisión hospitalaria, para identificar patrones iniciales que puedan aportar información útil al proceso de modelado.

En primer lugar, se analiza la relación entre variables clínicas generales como la edad (age), el tiempo de hospitalización (time_in_hospital), el número de procedimientos de laboratorio (num_lab_procedures) y el número de medicamentos administrados (num_medications) frente a la variable objetivo. Mediante el uso de un pairplot, se visualizarán estas comparaciones, representadas en la siguiente figura:



Tras la observación del gráfico, no se aprecia una relación clara ni una separación evidente entre clases utilizando estas características de forma individual o combinada. Esto sugiere que no existe una relación directa simple entre las variables analizadas y la variable objetivo, por lo que resulta necesario recurrir a técnicas de aprendizaje automático capaces de capturar relaciones no lineales y dependencias más complejas entre variables.

Posteriormente, se estudian variables relacionadas con la utilización de recursos sanitarios previos, como el número de ingresos hospitalarios (`number_inpatient`), visitas a urgencias (`number_emergency`) y consultas ambulatorias (`number_outpatient`), así como el número de diagnósticos (`number_diagnoses`).



En este caso, se observa que los pacientes con mayor número de ingresos hospitalarios previos suelen ser casos de readmisión, lo cual es coherente desde un punto de vista clínico, ya que puede indicar una mayor gravedad o cronicidad de la enfermedad.

En conjunto, aunque no se identifican separaciones claras entre clases mediante combinaciones simples de variables, sí se observan tendencias que sugieren cierta relación entre la carga clínica del paciente y la probabilidad de readmisión. Estos resultados refuerzan la necesidad de emplear

algoritmos de aprendizaje automático más avanzados, así como técnicas de tratamiento del desbalance previamente descritas, dado que las relaciones observadas no son triviales ni fácilmente separables mediante reglas simples.

4.3. Baseline (DummyClassifier)

En este apartado se implementará un baseline (punto de referencia) mediante un DummyClassifier, que sirve para establecer un nivel mínimo de rendimiento frente al cual comparar los modelos más complejos, generando predicciones simples basadas en estrategias como la clase mayoritaria, sin tener en cuenta la información de las variables de entrada [3], obteniendo los siguientes resultados de accuracy, balanced accuracy y AUC:

Accuracy	Balanced Accuracy (10 CV)	AUC (10 CV)
0,886	0,500	0,500

Como se ha mencionado antes, no nos podemos fiar del accuracy ya que la alta precisión que tiene es causada por el desbalance de las clases target. Por otro lado, el balanced accuracy 10 CV y AUC 10 CV son los esperados, 0.5 lo cual es lo esperado. Estos resultados confirman que el modelo no tiene capacidad predictiva real, estableciendo así una referencia mínima que los modelos posteriores deberán superar.

5. Modelado inicial

Para comenzar con los primeros modelos de aprendizaje, se usarán los que probablemente serán los más sencillos para predecir. En concreto, se han seleccionado los algoritmos de K-Nearest Neighbors (KNN) y Regresión Logística.

5.1. K-Nearest Neighbors

El KNN (o k-vecinos más cercanos) que clasifica nuevos puntos de datos basándose en la proximidad (distancia) a los "k" puntos más cercanos en el conjunto de entrenamiento, asumiendo que elementos similares están cerca [4]. Obteniendo los siguientes valores:

Accuracy (10 CV)	Balanced Accuracy (10 CV)	AUC (10 CV)
0,878	0,512	0,555

Como se ve, presenta una Accuracy sesgada por el desbalance, por otro lado, la Balanced Accuracy y AUC superan ligeramente el del DummyClassifier, indicando que capta cierta información de las variables de entrada, pero su capacidad predictiva sigue siendo limitada probablemente debido a las limitaciones inherentes del algoritmo y a la naturaleza del problema.

5.2. Regresión logística

Por otro lado, la Regresión Logística es un modelo lineal utilizado comúnmente en problemas de clasificación binaria, calculando una probabilidad y clasificando si pertenece a una clase u otra dependiendo de si supera o no cierto umbral [5].

A diferencia de modelos como KNN, este algoritmo aprende una frontera de decisión global, lo que lo hace más estable en presencia de ruido y alta dimensionalidad [6]. Además, se ha empleado el parámetro “class_weight='balanced'” para compensar el desbalance de la variable objetivo.

Accuracy (10 CV)	Balanced Accuracy (10 CV)	AUC (10 CV)
0,660	0,612	0,654

Como se observa, presenta una accuracy inferior a modelos anteriores, lo cual es esperable debido a la penalización del desbalance. Sin embargo, la balanced accuracy y el AUC muestran una mejora notable respecto al baseline y al modelo KNN, alcanzando valores de 0.612 y 0.654 respectivamente. Esto indica que el modelo es capaz de discriminar de forma más efectiva entre pacientes readmitidos y no readmitidos.

Estos resultados posicionan a la Regresión Logística como un modelo base sólido, sirviendo como referencia frente a modelos más complejos que se evaluarán posteriormente.

6. Gestión del desbalance, SMOTE

SMOTE (Synthetic Minority Over-sampling Technique) es una técnica de aumento de datos que soluciona el desequilibrio de clases creando ejemplos nuevos y sintéticos para la clase minoritaria, en lugar de duplicar casos [8]. En este apartado se aplicó usando el AUC como métrica para el mejor modelo de los anteriores, la Regresión Logística. Sin embargo, los resultados muestran una ligera disminución del rendimiento al aplicar esta técnica.

Tras este resultado inesperado, para comprobar su funcionamiento también se usó con el modelo KNN, obteniendo un valor esta vez coherente como se ve en la siguiente Figura:

	KNN	Logistic Regression
Sin SMOTE	0,555	0,654
SMOTE	0,559	0,645

Como se observa en la tabla, el comportamiento de SMOTE difiere según el modelo utilizado. En el caso de KNN, el uso de sobremuestreo produce una ligera mejora en el AUC, pasando de 0.555 sin SMOTE a 0.559 con este. Esto sugiere que la generación de muestras artificiales ayuda a mejorar la representación de la clase minoritaria beneficiando a los modelos basados en distancias como KNN.

Sin embargo, en la Regresión Logística se observa una disminución, pasando de 0.654 sin SMOTE a 0.645 con SMOTE. Este resultado indica que, en este caso, la técnica de ponderación de clases mediante “class_weight='balanced'” es suficientemente efectiva para tratar el desbalance, además no añade ruido como el SMOTE, que empeoró el AUC en la prueba anterior.

Por este motivo, en los modelos siguientes y que lo permitan, se priorizará el uso de class_weight='balanced' como estrategia de gestión del desbalance, dada su mejor consistencia en los resultados obtenidos en este conjunto de datos.

Cabe destacar que al crear muestras sintéticas basadas en otras muestras del dataframe, puede haber riesgo de “data leakage”, que ocurre cuando información del conjunto de test se filtra en los modelos, permitiendo que el modelo se entrene con información no disponible en la fase de entrenamiento [9]. En el caso del sobremuestreo, este problema aparecería si SMOTE se aplicará antes de la partición de los datos, ya que las muestras sintéticas podrían generarse utilizando información de todo el dataset, incluyendo el conjunto de validación.

Para evitar esto, se ha aplicado dentro de un Pipeline de la librería imbalanced-learn (imblearn) sobre los datos de entrenamiento en cada fold de la validación cruzada, asegurando que los datos sintéticos no se generen a partir de información del conjunto de test.

7. Modelado intermedio

Una vez evaluados los modelos iniciales, se ampliará el análisis incorporando nuevos algoritmos de clasificación con enfoques de aprendizaje distintos. En concreto, se evaluarán tres modelos adicionales:

7.1. DecisionTreeClassifier

Los Árboles de Decisión para la clasificación son algoritmos que crean decisiones mediante estructuras jerárquicas similares a diagramas de flujo, dividiendo datos en subconjuntos homogéneos [10]. Se usó DecisionTreeClassifier de SciKit Learn obteniendo los siguientes resultados:

Accuracy (10 CV)	Balanced Accuracy (10 CV)	AUC (10 CV)
0,629	0,613	0,647

Tras la evaluación, podemos concluir que este modelo es sólido para las predicciones, sin embargo, es inferior a el modelo Logistic Regression, superando únicamente la métrica de balanced accuracy en una diferencia mínima de 0.001.

7.2. RandomForestClassifier

El Random Forest Classifier (Clasificador de Bosque Aleatorio) es un algoritmo que crea múltiples árboles de decisión entrenados con diferentes subconjuntos de datos, combinando sus predicciones (mediante una votación) para obtener un resultado final más preciso, estable y menos propenso al overfitting que un solo árbol [11], por lo que podemos esperar un rendimiento mejor a su variante más simple, el Árbol de Decisión.

Accuracy (10 CV)	Balanced Accuracy (10 CV)	AUC (10 CV)
0,642	0,618	0,661

Como era de esperar, este modelo ha superado con creces los valores del modelo anterior y también los del Logistic Regression. En particular, se observa una mejora en el AUC, alcanzando un valor de 0.661, lo que indica una mayor capacidad del modelo para discriminar entre pacientes readmitidos y no readmitidos.

Este incremento en el rendimiento puede explicarse por la naturaleza del algoritmo, ya que al combinar múltiples árboles de decisión es capaz de capturar relaciones no lineales e interacciones complejas entre las variables, reduciendo además el riesgo de sobreajuste presente en un único árbol.

En conjunto, el Random Forest se posiciona como el mejor modelo evaluado hasta el momento, mostrando un equilibrio adecuado entre capacidad predictiva y estabilidad, por lo que será un buen candidato para aplicar técnicas de optimización en las siguientes etapas del proyecto.

7.3. ADABOOST

Finalmente, se probó AdaBoost (Adaptive Boosting o Refuerzo Adaptado), su objetivo principal es mejorar la precisión de modelos de clasificación simples (denominados "aprendices débiles" o weak learners) combinándolos para crear un modelo sólido y "fuerte" [12]. En este ejemplo, se realizó usando 10 de estos estimadores débiles obteniendo los siguientes pesos y su Balanced Accuracy y AUC.

nº	Peso
1	3,89E-01
2	1,33E+00
3	8,27E-04
4	4,44E-16
5	0,00E+00
6	0,00E+00
7	0,00E+00
8	0,00E+00
9	0,00E+00
10	0,00E+00

Balanced Accuracy (10 CV)	AUC (10 CV)
0,587	0,627

El análisis de los clasificadores débiles muestra que los pesos se concentran en las primeras iteraciones, mientras que los modelos posteriores (a partir del quinto) no aportan contribuciones, lo que sugiere una capacidad limitada del algoritmo para seguir mejorando el rendimiento en este conjunto de datos.

Además, esto explica sus métricas obtenidas, con una Balanced Accuracy de 0.587 y un AUC de 0.627 posicionándose por debajo de otros modelos.

En conclusión, Random Forest obtiene el mejor rendimiento entre los modelos anteriores, lo que sugiere que en este conjunto de datos el enfoque basado en bagging (entrenamiento en paralelo de múltiples modelos sobre distintas muestras de los datos y combinando sus predicciones [13]) es más efectivo que técnicas de boosting como del AdaBoost (entrenamiento secuencial donde cada modelo corrige los errores del anterior [14]), ya que permite reducir la varianza y mejorar la estabilidad del modelo frente al desbalance y la complejidad de los datos [15].

8. Selección de características

Tras la evaluación de diferentes modelos de aprendizaje automático y la obtención de un rendimiento estable, se aborda la etapa de selección de características para analizar la relevancia de las variables del conjunto de datos.

8.1. Algoritmos de selección de características

Para obtener dichos rasgos se usaron 2 algoritmos distintos, el primero de ellos RFE (del inglés Eliminación Recursiva de Características) un algoritmo que entrena un modelo, calcula la importancia de cada variable y elimina la menos relevante, después, se repite el proceso hasta obtener las variables deseadas [16].

Por otro lado, también se usó SelectKBest, un método de selección univariante que evalúa la relación estadística entre cada variable y la variable objetivo de forma independiente. En este caso, al ser un problema de clasificación, se empleó la función “f_classif” basada en el análisis ANOVA, que mide si existen diferencias significativas entre las clases para cada variable. [17]

Ambos métodos permiten reducir la dimensionalidad del problema, eliminando variables con menor relevancia predictiva y facilitando la interpretación del modelo, además de reducir el riesgo de sobreajuste.

8.2. Discusión de resultados

Los resultados obtenidos después de la ejecución de ambos algoritmos son recogidos en la siguiente tabla, ordenada del 1 al 10 en cuanto a su importancia. Esta comparación sirve para comprobar la consistencia entre ambos métodos y la coherencia de las predicciones en base a una perspectiva clínica.

Orden de importancia	RFE	SelectKBest
1	age	time_in_hospital
2	time_in_hospital	num_medications
3	num_lab_procedures	number_emergency
4	num_procedures	number_inpatient
5	num_medications	number_diagnoses
6	number_outpatient	metformin
7	number_emergency	insulin
8	number_inpatient	diabetesMed
9	number_diagnoses	diag_2_Neoplasias
10	discharge_disposition_id_Traslado	discharge_disposition_id_Traslado

Como se aprecia en la tabla, las variables sombreadas en verde corresponden a aquellas que coinciden entre ambos métodos de selección de características. Estas son, en su mayoría, variables que describen visitas previas al centro sanitario como: *time_in_hospital*, *number_inpatient*, *number_emergency* y *discharge_disposition_id*. Las otras variables coincidentes son *num_medications* y *number_diagnoses*, relacionadas con la complejidad de la enfermedad.

De forma individual, RFE capta variables como *age* o *num_lab_procedures* siendo la edad a la que más importancia le da. Por otro lado, SelectKBest se centra más en medicación, con variables como *metmorfin* o *insulin* y considera importante para la predicción un diagnóstico secundario relacionado con las neoplasias con *diag_2_Neoplasias*.

De forma conjunta, ambos métodos muestran unos resultados coherentes; las coincidencias en variables que están relacionadas con la utilización previa de recursos sanitarios y con la complejidad del paciente ponen de manifiesto la importancia de dichas variables. Mientras que las diferencias observadas muestran que cada uno de los métodos ha sido capaz de captar distintas facetas del problema, ya que RFE incluye variables más globales (la edad o el número de pruebas) y SelectKBest selecciona diagnósticos y tratamientos.

En términos clínicos esta combinación tiene sentido, dado que el riesgo de readmisión depende del historial del paciente, del estado clínico y del tratamiento. Por tanto se puede concluir que la selección de características resulta adecuada y coherente.

9. Optimización de hiperparámetros

Hasta ahora, los modelos de aprendizaje usados fueron ajustados con parámetros por defecto o manualmente. Esta práctica no es óptima ya que el rendimiento de los algoritmos está estrechamente ligado a la configuración de sus hiperparámetros.

Por este motivo, en esta sección se aborda la optimización automática con el objetivo de mejorar el rendimiento de los modelos y encontrar configuraciones más adecuadas para la predicción.

9.1. GridSearchCV

Esta herramienta nativa de SciKit Learn realiza una búsqueda de los mejores hiperparámetros dada una cuadrícula (Grid) predefinida, en la que se especifican los parámetros a optimizar y los valores que se desean evaluar para cada uno de ellos [18].

Sin embargo, este enfoque presenta limitaciones en términos de eficiencia computacional, requiriendo mucho tiempo para buscar los mejores valores.

Para probar la herramienta, se usó el modelo con el mejor rendimiento actualmente, el RandomForest, obteniendo sus mejores parámetros de los parámetros *n_estimators*, *max_depth* y *min_samples_split*, además de *class_weight='balanced'* por su gran rendimiento:

```
{'class_weight': 'balanced', 'max_depth': 10, 'min_samples_split': 5, 'n_estimators': 200}
```

Asimismo, para poder comparar la herramienta, se buscó una de mayor nivel como Optuna, basada en una búsqueda adaptativa de hiperparámetros.

9.2. Optuna

Esta herramienta de código abierto desarrollada por Preferred Networks (PFN), explora de forma inteligente el espacio de búsqueda, descartando combinaciones poco prometedoras y centrándose en aquellas con mayor potencial de mejora, lo que permite encontrar soluciones óptimas de manera eficiente. Para el mismo modelo, se ejecuta definiendo como espacio de búsqueda los hiperparámetros *n_estimators*, *max_depth* y *min_samples_split*. Adicionalmente, se mantuvo el parámetro *class_weight='balanced'*, dado su buen comportamiento en el tratamiento del desbalance de clases observado en apartados anteriores. Los resultados son los siguientes:

```
{'n_estimators': 633, 'max_depth': 9, 'min_samples_split': 4}
```

9.3. Resultados y discusión

Con estos mejores hiperparámetros, entrenamos dos RandomForest y obtenemos sus métricas, además de compararlas con el modelo entrenado anteriormente a modo de referencia (*benchmark*).

	Accuracy (10 CV)	Balanced Accuracy (10 CV)	AUC (10 CV)
Random Forest base	0,642	0,618	0,661
GridSearch	0,674	0,615	0,663
Optuna	0,660	0,617	0,663

En conclusión, la optimización de hiperparámetros produce una mejora leve frente al modelo base de Random Forest. En términos de accuracy, tanto GridSearch como Optuna superan al modelo original (0,674 y 0,660 frente al 0,642), lo que pone de manifiesto una mejora en las capacidades globales de

clasificación. Sin embargo, al utilizar la métrica balanced accuracy el modelo base queda en cabeza en la capacidad de clasificación (0,618), seguido muy de cerca por Optuna (0,617) y con GridSearch en el último lugar (0,615).

La métrica AUC se muestra similar, con ambos métodos optimizados nos acercamos a los 0,663 de AUC, ligeramente mejor que el 0,661 del modelo base, aunque la diferencia al igual que antes no es concluyente.

En definitiva, los resultados de la optimización de hiperparámetros no suponen una mejora importante respecto al rendimiento del Random Forest para este conjunto de datos. Sin embargo, Optuna produce un rendimiento comparable a GridSearch con una exploración del espacio de búsqueda más eficiente, aunque con un coste temporal reducido.

10. Modelado avanzado

Una vez analizados y optimizados algunos modelos, se procede al uso de un modelo avanzado considerado idóneo para datos tabulares.

Inicialmente, se evaluó XGBoost como alternativa dentro de los métodos de gradient boosting, aunque su rendimiento no superó al obtenido por los mejores modelos previos, por lo que no se continuó su análisis en profundidad. [20]

Posteriormente, se empleó LightGBM, un algoritmo de boosting más eficiente y ligero, que mostró un mejor comportamiento en este conjunto de datos por lo que se continuó evaluando obteniendo los siguientes resultados:

Accuracy (10 CV)	Balanced Accuracy (10 CV)	AUC (10 CV)
0,645	0,617	0,663

Como vemos, este modelo obtuvo un rendimiento igual de Balanced Accuracy y AUC que el RandomForest optimizado por Optuna, por lo que se optó por optimizar con la misma herramienta este modelo en búsqueda de una mejora.

Al re-entrenar el modelo con sus mejores parámetros en 50 *trials*:

```
{'n_estimators': 443, 'learning_rate': 0.017585648705224517, 'num_leaves': 100, 'max_depth': 4,  
  'min_child_samples': 32}
```

Obtenemos los siguientes valores de métricas:

	Accuracy (10 CV)	Balanced Accuracy (10 CV)	AUC (10 CV)
Random Forest Optuna	0,660	0,617	0,663
LightGBM	0,645	0,617	0,663
LightGBM Optuna	0,639	0,619	0,664

En general, los resultados no muestran casi diferencias en los modelos evaluados, sin un claro dominador en todas las métricas. Mientras que el Random Forest optimizado con Optuna presenta el mejor valor de *accuracy*, el modelo LightGBM optimizado obtiene ligeramente mejores resultados en *balanced accuracy* y *AUC*, métricas más relevantes en problemas con desbalance de clases.

Por tanto, ambos modelos muestran un rendimiento comparable, destacando LightGBM Optuna como una alternativa ligeramente más adecuada desde el punto de vista del equilibrio entre clases.

11. Comparación estadística

Dado el conjunto de modelos evaluados hasta el momento, se presenta a continuación una tabla comparativa con sus principales métricas obtenidas mediante validación cruzada:

	Accuracy (10 CV)	Balanced Accuracy (10 CV)	AUC (10 CV)
DummyClassifier	0,886	0,500	0,500
KNN	0,878	0,512	0,555
Linear Regression	0,660	0,612	0,654
Decision Tree	0,629	0,613	0,647
Random Forest base	0,642	0,618	0,661
Random Forest GridSearch	0,674	0,615	0,663
Random Forest Optuna	0,660	0,617	0,663
LightGBM	0,645	0,617	0,663
LightGBM Optuna	0,639	0,619	0,664

Aunque el DummyClassifier presenta el mayor valor de *accuracy*, este resultado ya ha sido explicado previamente y se debe al fuerte desbalance de clases del conjunto de datos, por lo que no es una métrica representativa del rendimiento real del modelo.

Por lo tanto, LightGBM Optuna será el candidato para el análisis estadístico por ser el modelo con mayor *Balanced Accuracy* y *AUC* (aunque por muy poco).

A continuación, se procede a aplicar una prueba estadística para determinar si dichas diferencias son significativas o si pueden atribuirse al azar. En concreto, se utiliza el test no paramétrico de rangos con signo de Wilcoxon sobre los resultados de *Balanced Accuracy* y *AUC* obtenidos mediante validación cruzada.

Comparación LGBM Optuna	Estadístico	p-valor	Significativo
vs DummyClassifier	55	0,001	SÍ
vs KNN	55	0,001	SÍ
vs LogisticRegression	44	0,0527	NO
vs Árbol	53	0,0029	SÍ
vs AdaBoost	55	0,001	SÍ
vs Random Forest	35	0,2461	NO
vs RF GridSearchCV	43	0,0654	NO
vs RF Optuna	40	0,1162	NO
vs LGBM	35	0,2461	NO

Test de Wilcoxon según su Balanced Accuracy

Comparación LGBM Optuna	Estadístico	p-valor	Significativo
vs DummyClassifier	55	0,001	SÍ
vs KNN	55	0,001	SÍ
vs LogisticRegression	44	0,0527	NO
vs Árbol	55	0,001	SÍ
vs AdaBoost	55	0,001	SÍ
vs Random Forest	52	0,0049	SÍ
vs RF GridSearchCV	44	0,0527	NO
vs RF Optuna	40	0,1162	NO
vs LGBM	31	0,3848	NO

Test de Wilcoxon según su AUC-ROC

Los resultados muestran que este modelo es significativamente superior a los modelos más simples como el DummyClassifier, KNN, Árbol de decisión y AdaBoost ($p < 0.05$), lo que indica una mejora clara respecto a enfoques básicos.

Sin embargo, no hay diferencias estadísticamente significativas frente a los modelos más complejos como LightGBM base, Random Forest optimizado o Logistic Regression (aunque se queda muy cerca de ser significativo en ambas métricas). Únicamente en el caso de Random Forest base se aprecia una diferencia significativa con la métrica AUC-ROC, aunque este resultado no se cumple igual con la otra métrica.

En conclusión esto indica que, aunque LightGBM optimizado mediante Optuna obtiene el mejor rendimiento medio, las diferencias con otros modelos avanzados no son estadísticamente significativas. Aun así, este modelo se utilizará como referencia para las siguientes etapas del análisis debido a su rendimiento competitivo y su comportamiento consistente en las métricas evaluadas.

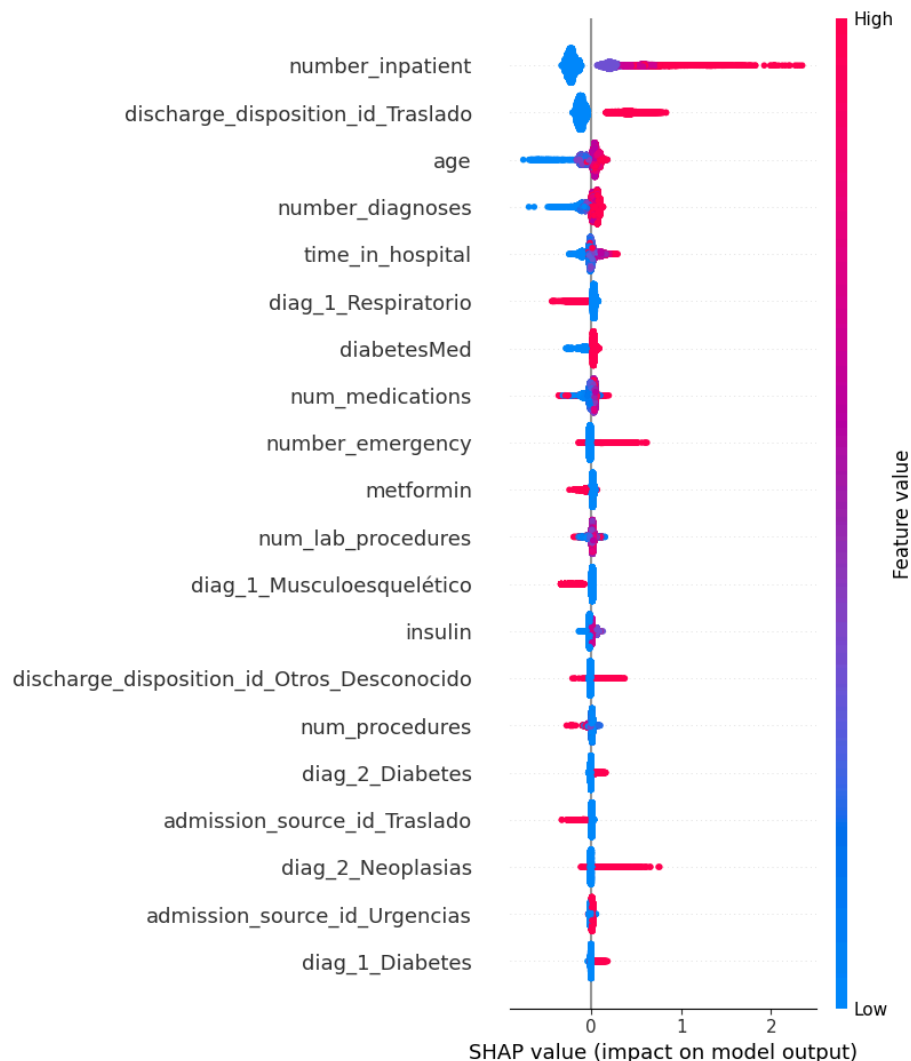
12. Explicabilidad (XAI)

Una vez elegido nuestro modelo final, no podemos tratarlo como una caja negra que devuelve predicciones sin ningún motivo, por esa razón usaremos 2 librerías de explicabilidad para proporcionar una visión más transparente de su funcionamiento.

12.1. SHAP

En primer lugar, se emplea SHAP, una técnica basada en teoría de juegos que permite cuantificar la contribución de cada variable a las predicciones del modelo. SHAP da a cada característica un valor de importancia y analiza su influencia tanto globalmente como para muestras en particular. [21]

A continuación, se muestra el resumen global de importancia de variables obtenido:

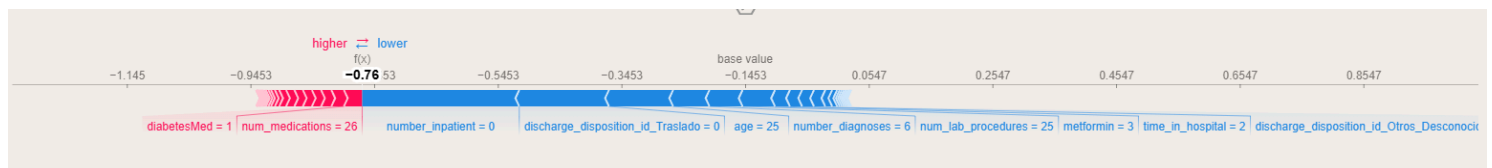


Dada la gráfica anterior, podemos concluir que la variable más influyente en el modelo es *number_inpatient*, lo que indica que cuantos más ingresos haya tenido un paciente, mayor será la probabilidad de que vuelva a ingresar. Esto muestra que la cronicidad del paciente es muy importante.

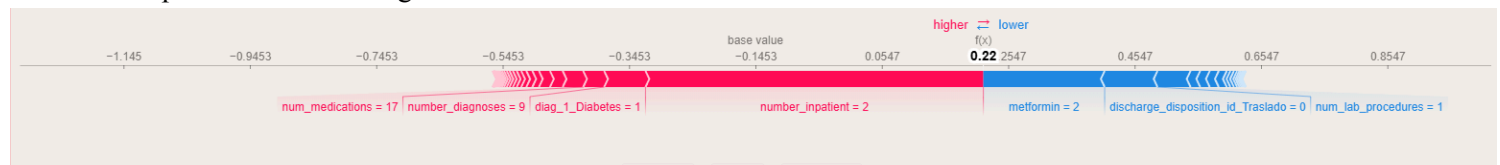
También se destacan algunas variables clínicas y de gestión hospitalaria. Por ejemplo, el traslado del paciente después de ser dado de alta y ciertos diagnósticos, como neoplasias y problemas respiratorios, se asocian con un mayor riesgo de reingreso.

Además, factores como la edad y el número de diagnósticos también contribuyen positivamente a la predicción. En general, el modelo LightGBM parece haber aprendido patrones que tienen sentido desde el punto de vista clínico.

Además, se puede obtener un gráfico individual para cada predicción, se representará uno de un paciente NO readmitido y uno readmitido respectivamente:



En este gráfico, se aprecia una probabilidad baja de reingreso (-0.76), con las variables que bajan la probabilidad de reingreso en azul.



Al contrario de este caso, predominan variables rojas que “empujan” la predicción hacia el positivo (0.22) lo cual significa que el modelo encuentra un riesgo de readmisión del paciente.

12.2. LIME

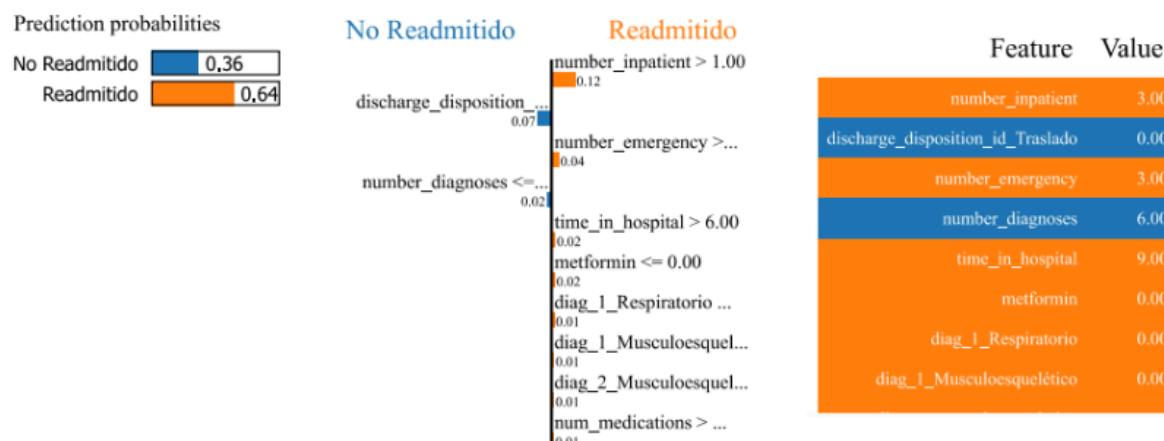
LIME (Local Interpretable Model-agnostic Explanations) es una técnica de explicabilidad que se utiliza para interpretar predicciones individuales, en la práctica, genera una gráfica donde se muestran las características que más han influido en una predicción individual, indicando si cada variable empuja la predicción hacia una clase u otra. [22]

Obtenemos un gráfico para cada caso:



El modelo predice NO readmitido con una probabilidad del 59%. LIME muestra que la ausencia de ingresos previos (*number_inpatient*) y la falta de traslados en el alta son los factores más

determinantes para esta predicción, favoreciendo la clase negativa y compensando leves señales de riesgo como el número de diagnósticos.

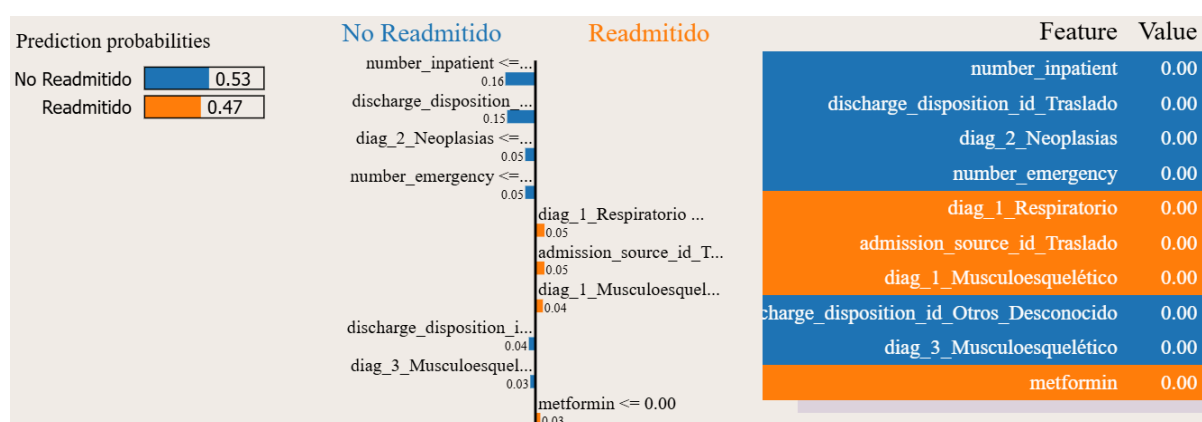


Por otro lado, en un caso clasificado como Readmitido, con una probabilidad del 64%, se observa que un mayor número de ingresos previos, la presencia de visitas a urgencias y una estancia hospitalaria prolongada actúan como principales factores que empujan la predicción hacia la clase positiva.

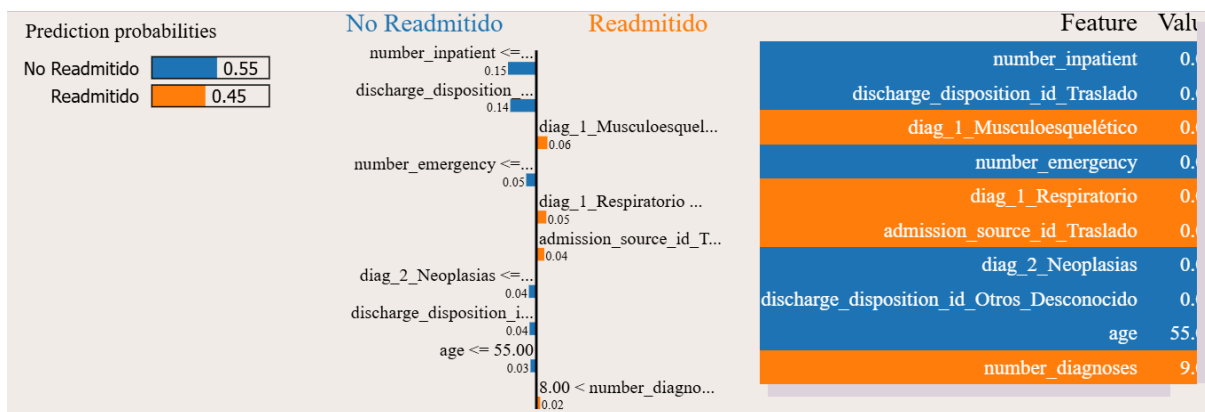
De este apartado podemos concluir que el modelo basa sus predicciones en variables relevantes que se relacionan con la gravedad y la complejidad del estado del paciente.

13. Análisis de errores

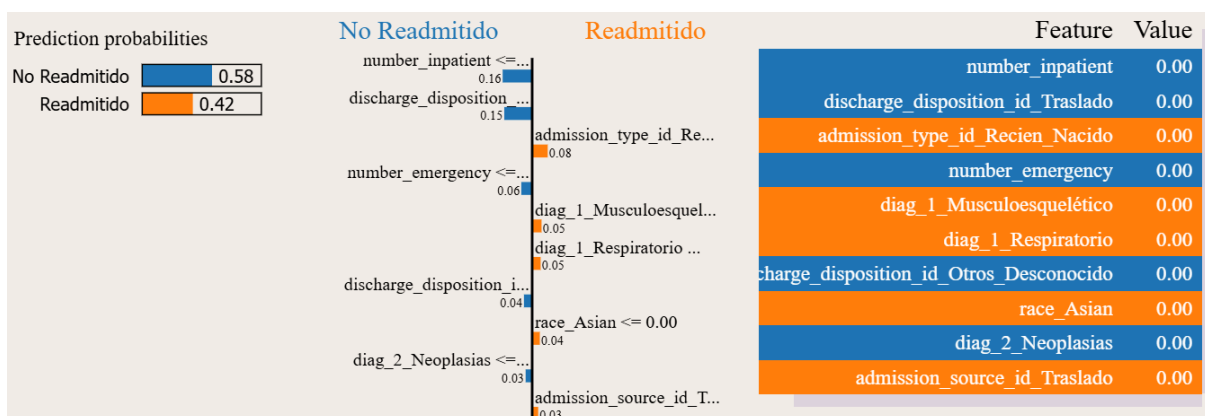
Con el objetivo de evaluar el comportamiento del modelo en situaciones incorrectas, se seleccionaron tres casos en los que la predicción no coincide con la etiqueta real. Para esto, se buscó en el dataframe los valores predichos como NO readmitido ($predicción == 0$) y realmente fueron readmitidos ($real == 1$). De esta forma se obtuvieron los siguientes casos, los cuales fueron graficados en un LIME para encontrar una posible explicación:



ID 39110



ID 43788



ID 49642

Tras el análisis del *LIME*, se observa un patrón claro en el comportamiento del modelo. En particular, variables como *number_inpatient* = 0 y *discharge_disposition_id_Traslado* = 0 (alta a domicilio) tienden a inclinar fuertemente la predicción hacia la clase *No Readmitido*, lo que indica una fuerte dependencia del historial previo del paciente.

Para complementar este análisis, se hará una evaluación del modelo en función de estas variables problemáticas, comprobando sobre los reingresos reales los falsos negativos por culpa de estas variables. En la siguiente tabla, se recoge el porcentaje de falsos negativos cuando se cumple o no la condición problemática:

	<i>number_inpatient</i> = 0	<i>discharge_disposition_id_Traslado</i> = 0 (Alta)
SI	67,23%	55,11%
NO	14,77%	9,33%

Por lo que podemos concluir que el modelo es fiable con pacientes conocidos (15% de error), pero sufre de una ceguera sistémica ante pacientes sin ingresos previos, fallando en el 67% de

los casos porque depende excesivamente del historial para activar la alerta de riesgo como se comprobó en el apartado de selección de características. Por otro lado, cuando se le dió el alta al paciente, falla la mitad de las veces mientras que solo se equivoca un 9% cuando el paciente fue trasladado.

14. Fairness

Finalmente, con el objetivo de evaluar la robustez del modelo, se analiza su comportamiento en diferentes subgrupos demográficos, incluyendo género, raza y edad. Evaluando la tasa de falsos negativos en dichos grupos, ya que en este problema clínico es especialmente relevante no omitir pacientes con riesgo de reingreso.

Los resultados obtenidos con el mismo algoritmo que el apartado 13, concluyó con los siguientes resultados:

FALLOS POR SUBGRUPO (género, raza y edad):

----- Género:

Tasa de fallo (Falsos Negativos) en Hombres: 41.88%

Tasa de fallo (Falsos Negativos) en Mujeres: 40.23%

Raza:

Tasa de fallo (Falsos Negativos) en Asiáticos: 31.25%

Tasa de fallo (Falsos Negativos) en Caucásicos: 40.79%

Tasa de fallo (Falsos Negativos) en Hispanos: 30.30%

Tasa de fallo (Falsos Negativos) en Otros: 51.52%

Tasa de fallo (Falsos Negativos) en Afroamericanos: 42.15%

Edad:

Tasa de fallo (Falsos Negativos) en Mayores de 70: 36.56%

Tasa de fallo (Falsos Negativos) en Jóvenes/Mediana edad: 44.80%

En el caso del género, no se observan diferencias relevantes entre hombres y mujeres, con tasas de error del 41,88% y 40,23% respectivamente, lo que indica un comportamiento equilibrado.

Respecto a la raza, se observan ligeras variaciones entre grupos. Destaca un mayor error en el grupo “Otros” (51,52%), mientras que asiáticos e hispanos presentan tasas más bajas (31,25% y 30,30%) muy posiblemente ligado a la baja representación en el dataset. Los grupos mayoritarios, como caucásicos y afroamericanos, se sitúan en valores intermedios, alrededor del 40%.

En relación con la edad, se observa una diferencia más marcada. Los pacientes mayores de 70 años presentan una tasa de error del 36,56%, mientras que en pacientes más jóvenes o de mediana edad el error aumenta hasta el 44,80%. Se comprobó la distribución indicando que el grupo de mayores de 70 años representa un ~45% del dataset, descartando hipótesis de infrarrepresentación como la raza Otros comentada anteriormente.

En conjunto, los resultados indican que el modelo presenta un comportamiento relativamente estable entre los distintos subgrupos analizados, salvo por alguna variable como la raza “Otros” o “Asiáticos” probablemente causada por una baja representación en el dataset.

15. Conclusiones

En este trabajo se ha abordado la construcción y evaluación de distintos modelos de aprendizaje automático para la predicción de reingresos hospitalarios, partiendo de modelos básicos hasta técnicas

avanzadas de boosting. Tras la comparación inicial, se observa que los modelos basados en ensembles, especialmente Random Forest y LightGBM, ofrecen un rendimiento superior frente a modelos más simples como KNN, Árbol de decisión o Regresión Logística.

La optimización con GridSearch y sobretodo Optuna permitió mejorar al máximo los modelos (aunque sin diferencias significativas entre ambos enfoques).

Posteriormente se realizó un test estadístico de Wilcoxon para comprobar la superioridad del modelo LightGBM optimizado con Optuna, que obtuvo las mejores medias de *AUC* y *Balanced Accuracy*. Aunque los resultados de esa prueba indicaron que el modelo es significativamente superior a los modelos simples, pero no consigue una significancia superior en modelos similares y optimizados.

En cuanto a la explicabilidad, el análisis mediante SHAP y LIME ha permitido interpretar tanto el comportamiento global como individual del modelo. Dando a entender el razonamiento de las predicciones, el cual ayudó para identificar los errores más típicos cometidos.

Finalmente, el análisis de equidad no muestra sesgos importantes en cuanto al género, la edad o la clase objetivo, lo que significa que el modelo se comporta de manera bastante estable en diferentes subgrupos, aunque hay algunas pequeñas discrepancias en razas infrarrepresentadas.

En general, podemos concluir que el modelo LightGBM es una buena opción para la predicción porque encuentra un buen equilibrio entre rendimiento, facilidad de comprensión y robustez. Esto lo hace adecuado para este tipo de predicción clínica, aunque todavía hay espacio para mejorar, especialmente en situaciones donde no hay mucha información clínica disponible.

16. Bibliografía

- [1] <https://allseniors.org/es/faqs/what-is-hospital-readmission-prevention/>
- [2] <https://archive.ics.uci.edu/dataset/296/diabetes+130-us+hospitals+for+years+1999-2008>
- [3] <https://interactivechaos.com/es/manual/tutorial-de-machine-learning/one-hot-encoding>
- [4] <https://www.ibm.com/es-es/think/topics/knn>
- [5] <https://www.ibm.com/es-es/think/topics/logistic-regression>

- [6] <https://www.geeksforgeeks.org/machine-learning/logistic-regression-vs-k-nearest-neighbors-in-machine-learning/>
- [7] <https://www.ibm.com/mx-es/think/topics/upsampling#:~:text=SMOTE.%20La%20t%C3%A9cnica%20de%20sobremuestreo%20sint%C3%A9tico%20de,los%20puntos%20existentes%20en%20la%20clase%20minoritaria.>
- [8] <https://learn.microsoft.com/es-es/azure/machine-learning/component-reference/smote?view=azureml-api-2>
- [9] <https://www.automatizapro.com.ar/blog/que-es-data-leakage-como-evitarlo/>
- [10] <https://www.uv.es/mlejarza/actuariales/tam/arbolesdecision.pdf>
- [11] https://es.wikipedia.org/wiki/Random_forest
- [12] <https://gamco.es/glosario/adaboost/>
- [13] <https://www.ibm.com/mx-es/think/topics/bagging#:~:text=Inspecci%C3%B3n%20visual-,%C2%BFQu%C3%A9%20es%20bagging?,un%20conjunto%20de%20datos%20ruidoso.>
- [14] <https://es.wikipedia.org/wiki/Boosting>
- [15] <https://medium.com/data-science-collective/bagging-vs-boosting-a-deep-dive-into-ensemble-learning-for-tree-based-models-b3a061da3e62>
- [16] https://qu4nt.github.io/sklearn-doc-es/modules/generated/sklearn.feature_selection.RFE.html
- [17] https://qu4nt.github.io/sklearn-doc-es/modules/generated/sklearn.feature_selection.SelectKBest.html
- [18] <https://keepcoding.io/blog/que-es-gridsearchcv/>
- [19] <https://en.wikipedia.org/wiki/Optuna>
- [20] https://medium.com/@priyaiotacademy122_2106/top-10-classification-algorithms-in-machine-learning-with-applications-b7d3736ab041
- [21] <https://www.datacamp.com/es/tutorial/introduction-to-shap-values-machine-learning-interpretability>
- [22] <https://medium.com/latinxinai/explicabilidad-de-modelos-de-ml-lime-f9d0dceb5154>